



Lecture 14: Binding Form Inputs

INTERVIEW QUESTION

[> CORE](#)

How do you bind form inputs to state?

Text inputs are easy, but checkboxes are booleans, radios pick one of many, and dropdowns can be multi-select. How do you bind each to state?

Building web forms in vanilla JavaScript is incredibly tedious. For every single input, you have to write an event listener to capture the user's keystrokes, and then write a separate function to push your data back into the input if it changes programmatically.

The Problem with One-Way Data Flow

Imagine you are building a complex user profile form with a text input for the username, a checkbox to accept terms, and a dropdown for their country.

If you just use `{value}` to set the input's value, data only flows in one direction: from your script to the DOM. When the user types into the input, the DOM changes, but your `$state` variable has no idea. To fix this, you have to attach an `oninput` handler to every single element to manually capture `event.target.value` and update your variables. This is boilerplate-heavy and quickly becomes unmanageable for large forms.

Two-Way Binding with `bind:`

Svelte solves this elegantly with the `bind:` directive. When you bind an input to a reactive variable, you establish a two-way connection. If the variable changes, the input updates. If the user types into the input, the variable updates automatically.



The image shows a code editor window titled "Profile.svelte". The code is as follows:

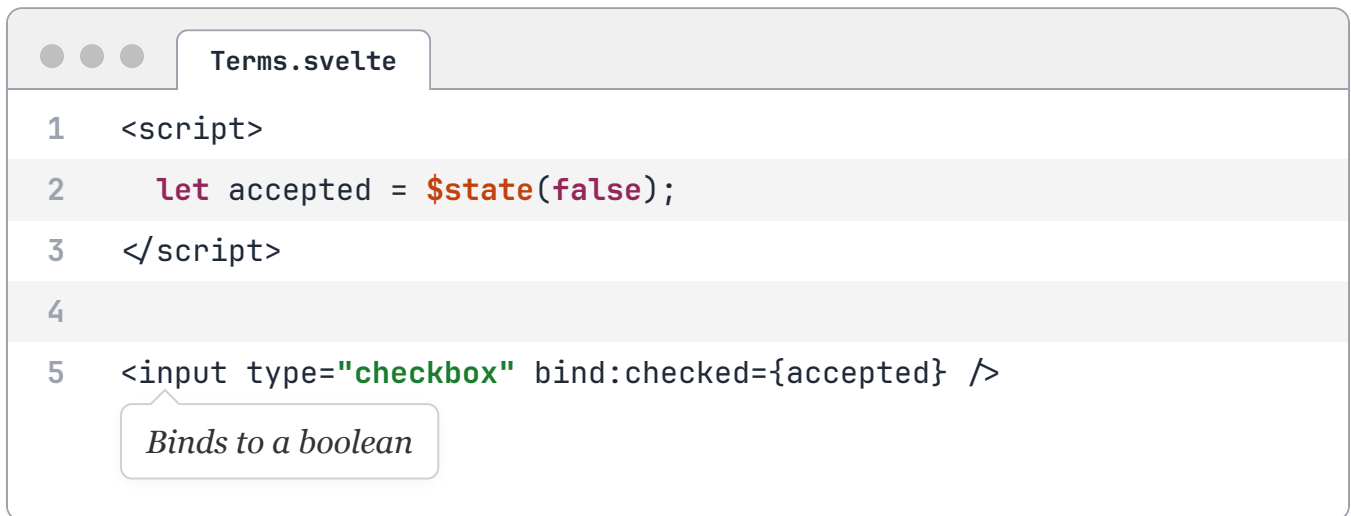
```
1 <script>
2   let username = $state("");
3 </script>
4
5 <input type="text" bind:value={username} />
6 <p>Hello, {username}</p>
```

A callout box points to the `bind:value={username}` attribute in line 5, containing the text *Two-way connection*.

Handling Different Input Types

The true power of Svelte's `bind:` directive is that it is smart enough to handle the idiosyncrasies of different HTML input types.

1. Checkboxes A checkbox doesn't have a string value; it is either checked or unchecked. You bind to its `checked` property using a boolean.



The screenshot shows a code editor window titled "Terms.svelte". It contains the following code:

```
1 <script>
2   let accepted = $state(false);
3 </script>
4
5 <input type="checkbox" bind:checked={accepted} />
```

A callout box points to the `bind:checked` attribute in line 5, containing the text "Binds to a boolean".

2. Radio Buttons Radio buttons work in groups where only one can be selected at a time. By binding all radios in a group to the same variable using `bind:group`, Svelte handles the mutual exclusivity for you.



The screenshot shows a code editor window titled "Survey.svelte". It contains the following code:

```
1 <script>
2   let difficulty = $state("easy");
3 </script>
4
5 <input type="radio" bind:group={difficulty} value="easy" />
6 <input type="radio" bind:group={difficulty} value="hard" />
```

3. Select Dropdowns For a standard `<select>` , you just bind to its `value` . For a `multiple` select, you bind to an array of values.

Country.svelte

```
1 <script>
2   let country = $state("US");
3 </script>
4
5 <select bind:value={country}>
6   <option value="US">United States</option>
7   <option value="UK">United Kingdom</option>
8 </select>
```

Value perfectly matches the selected option

KEY TAKEAWAY

Numeric inputs (`type="number"` or `type="range"`) are notoriously annoying in JavaScript because `event.target.value` is always a string. Svelte automatically handles type coercion for you: if you `bind:value` on a number input, your variable will be a true number, not a string.

Summary

Forms require two-way data flow to stay synchronized with your application state. Instead of writing manual event listeners, use Svelte's `bind:` directive. It handles text inputs (`bind:value`), checkboxes (`bind:checked`), radio groups (`bind:group`), and select dropdowns natively, completely abstracting away the tedious boilerplate of vanilla DOM events.

	INPUT TYPE	SVELTE DIRECTIVE
Text & Numbers	<code><input type="text"></code>	<code>bind:value</code>
Checkboxes	<code><input type="checkbox"></code>	<code>bind:checked</code>
Radio Buttons	<code><input type="radio"></code>	<code>bind:group</code>